> **Facilitator answer key — Holocron capstone worked solution.** Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

TMD-light — Holocron (release-1 slice)

Scope: source-string lifecycle + governed delivery (CS1–7, CS11–12, CS15). The fallback path carries the named invariant.

1. Data model

PostgreSQL system of record. Keys and versions **immutable** — an edit writes a new version row. Every mutating op writes an `AuditEntry`.

Entity	PK	Immutable?	Notable columns
Product	`product_id`	root mutable	`app_identifier` (namespace root), `target_languages[]`, `retired_at`
SourceString	`string_id`	key immutable	`namespace_key` (immutable, unique/product), `do_not_translate`, `criticality` (`normal` | `critical/legal`), `lifecycle_status`
SourceStringVersion	`src_version_id`	**yes**	`version_no`, `en_us_content`, `placeholder_meta`, `actor_id`, `created_at`
TranslationVariant	`variant_id`	container mutable	`locale`, `translation_state`, `published_selection_id`
TranslationVersion	`tr_version_id`	**yes**	`version_no`, `content`, `src_version_ref`, `actor_id`
ReviewRequest	`review_id`	append-only	`type` (Legal/Compliance/Peer/Translation), `status`, `outcome`
PublicationSelection	`selection_id`	**yes**	`resolved_source` (approved | previous | en-US-fallback), `tr_version_id?`, `published_at`
AuditEntry	`audit_id`	**yes**	`op`, `before`, `after`, `occurred_at` — mirrored to Event Hubs → Blob

Trade-off (named): `translation_state` is **denormalized** onto TranslationVariant so delivery needn't recompute "which version is live" per request; cost — publish/outdate must keep the pointer honest, enforced in one tx with the version insert. **Indexes (3):** `idx_delivery` on `SourceString(product_id, namespace_key) WHERE lifecycle_status='Published'` (partial, backs the delivery GET); `idx_variant_locale` on `TranslationVariant(string_id, locale)`; `idx_audit_entity_time` on `AuditEntry(entity_ref, occurred_at DESC)` (CS5).

2. Cloud topology

Layer	Azure service
Compute	**AKS** — management app + delivery API as separate deployments/HPAs
System of record	**Azure Database for PostgreSQL** (Flexible Server) — zone-redundant HA + read replica
Delivery cache	**Azure Cache for Redis** — keyed `product:namespace:locale`; publish invalidates
Delivery edge	**App Gateway + API Management** — TLS, rate limit (50 req/s, burst 100), token validation
Identity / secrets	**Entra ID** · **Key Vault**
Audit	**Event Hubs** → **Blob** (WORM archive)
Observability	**Azure Monitor**

Region: single primary, **zone-redundant** across 3 AZs; no active-active multi-region for R1 (99.9% met in-region; second region is an R2 option). **ROM:** ~**$4–6k/month** at R1 scale — delivery traffic, not the ~500k string volume, drives cost; Redis absorbs the read fan-out.

3. API contract

Method	Path	Auth	Codes	Idempotency
GET	`/v1/products/{productId}/namespaces/{namespace}?locale=…`	Entra bearer	200/401/403/404/429	pure read
POST	`…/namespaces/{namespace}/strings`	Content Owner	201/400/401/403/409	`Idempotency-Key`; duplicate key → 409
POST	`…/strings/{key}/versions`	Content Owner	201/…/409	edit = new immutable version

Method	Path	Auth	Codes	Idempotency
POST	`…/strings/{key}/review-requests`	Content Owner	201/…/409	one open request per (version, type)
PATCH	`…/review-requests/{id} {state:"complete"}`	Reviewer	200/…/404	idempotent transition
POST	`…/strings/{key}/publish`	Content Owner / Holocron Admin	200/…/409	PublicationSelection + AuditEntry in one tx; a `critical/legal` string needs a completed compliance review (else **409**) + synchronous cache invalidation (<5s)

Delivery GET returns only Published content for the namespace prefix, resolved for the locale:

```
[ {"key":"orders.checkout.submit","value":"Submit
order","locale":"ja-
JP","version":7,"fallback":false},

{"key":"orders.checkout.tax","value":"Tax","locale":
"en-US","version":3,"fallback":true} ]
```

`fallback:true` = no published translation for the locale, `en-US` served (CS7-AS-002). Management mutations are transactional; idempotency keys make POSTs replay-safe.

4. Sequence — publish → invalidate → fetch (with fallback)

Latencies annotate the **delivery read**; the publish write is bounded separately by the ≤60s propagation SLO.

```
PUBLISH (management, async to reads):
  POST /publish → Postgres: insert version + set
Published + AuditEntry (~15ms, one tx)
                → Redis DEL product:ns:* (~3ms) →
Event Hubs emit → Blob (async)
DELIVERY FETCH (consuming app, p95 ≤ 200ms):
  GET namespaces/{ns}?locale=ja-JP → APIM TLS +
token + rate check (~8ms)
    → Redis GET (~3ms) MISS → idx_delivery prefix
scan + PublicationSelection (~35ms)
    → resolve locale; no ja-JP translation → en-US,
fallback=true (~4ms) → Redis SET → 200
  ≈ 60ms cold / ~20ms warm — both inside p95 ≤ 200ms
```

Sad — over rate limit: 51st req/s → APIM returns **429 + Retry-After**; request never reaches AKS/Redis/Postgres. **Named invariant:** *the rate limiter is enforced at the edge, before any*

read path — a misbehaving consumer can't degrade latency for others, and no partial/uncached read is ever produced by a throttled request.

5. Performance baseline + monitoring

Baselines: delivery **p95 ≤200ms / p99 ≤500ms**; availability **99.9%** (management 99.5%); propagation **≤60s** normal / **<5s** `critical/legal` (synchronous, SEP §3).

SLO	Signal	Alert
Delivery latency	APIM/AKS duration histogram	p95 > 200ms or p99 > 500ms / 5 min
Delivery availability	APIM 5xx + synthetic probe	success < 99.9% / 30 min
Propagation	publish AuditEntry → first repopulated read	normal > 60s; `critical/legal` > 5s (tighter)
Rate limiting	APIM 429/consumer	sustained 429s → notify

Leading indicator (≤7 days): Redis cache-hit ratio — a falling ratio predicts rising p95 *before* the latency alert; dashboard it beside p95. **Also:** audit-lag, DB replica lag, fallback rate/locale (a spike = translations went Outdated after a source republish). **AI-summary note:** any AI dashboard summary is **advisory only**, validated against raw Azure Monitor metrics; it never gates an alert.